

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM

TIỂU LUẬN VERSION 1

Đề tài: Xây dựng Hệ thống E-Commerce theo Microservices và AI

Giảng viên hướng dẫn: PGS. TS. Trần Đình Quế

Nhóm lớp: 04 chính quy

Nhóm bài tập: 06

Sinh viên: Hoàng Lê Đức

Mã SV: B22DCCN224

Chương 1. Từ Monolithic đến Microservices và DDD

Chương 1 tập trung vào từ monolithic đến microservices và ddd. Nội dung được triển khai theo hướng từ khái niệm đến thiết kế thực hành, sau đó liên hệ trực tiếp với hệ thống ecom-final.

1.1. Giới thiệu Monolithic Architecture

Giới thiệu Monolithic Architecture là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần giới thiệu monolithic architecture nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

1.1.1. Khái niệm

Khái niệm là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

1.1.2. Cấu trúc điển hình

Cấu trúc điển hình là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

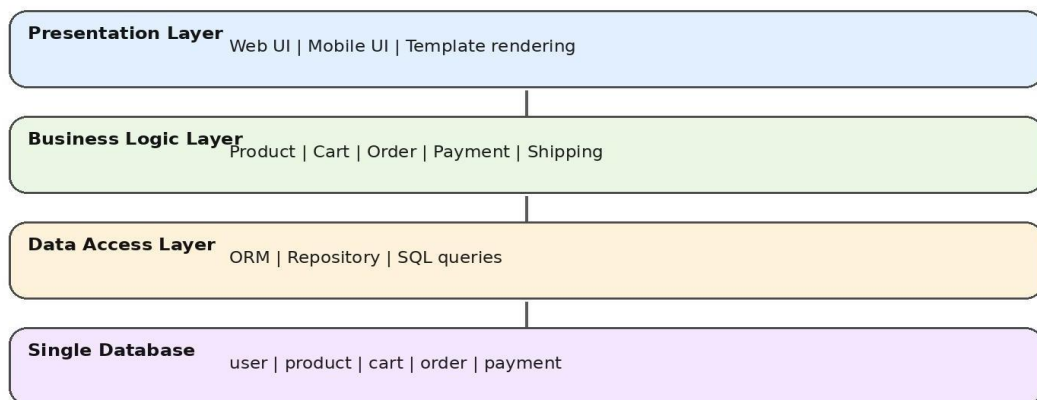
Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Kiến trúc Monolithic theo lớp



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

Hình 3. Cấu trúc monolithic

1.1.3. Ví dụ thực tế

Ví dụ thực tế là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần ví dụ thực tế nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

1.1.4. Nhược điểm chi tiết

Nhược điểm chi tiết là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần nhược điểm chi tiết nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

1.1.5. Khi nào nên dùng Monolithic

Khi nào nên dùng Monolithic là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

1.2. Microservices Architecture

Microservices Architecture là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

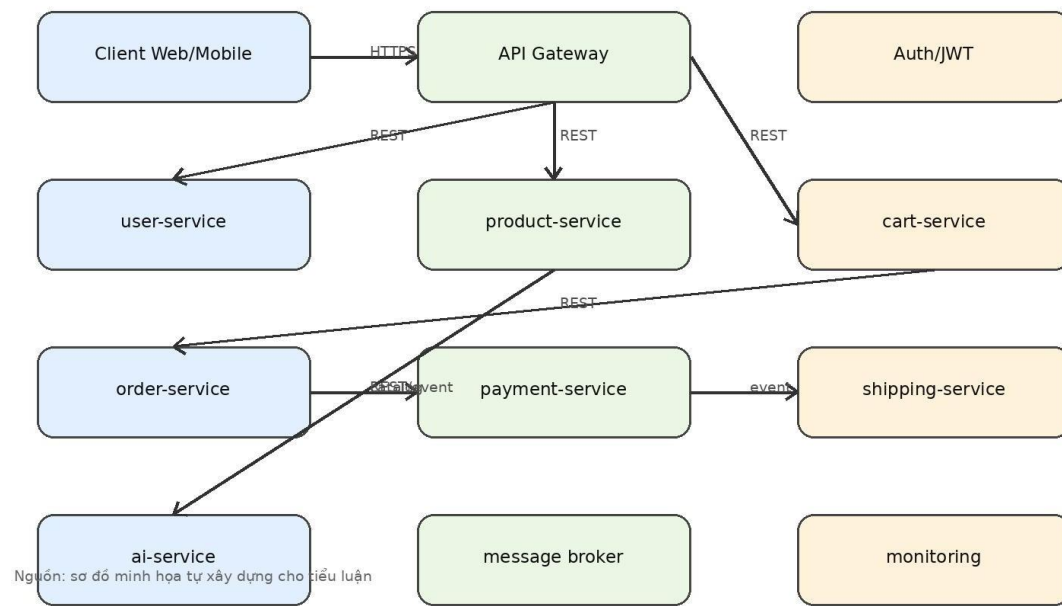
Trong tài liệu thiết kế, phần microservices architecture nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

Tổng quan Microservices cho E-Commerce



1.2.1. Khái niệm

Khái niệm là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

1.2.2. Đặc điểm

Đặc điểm là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

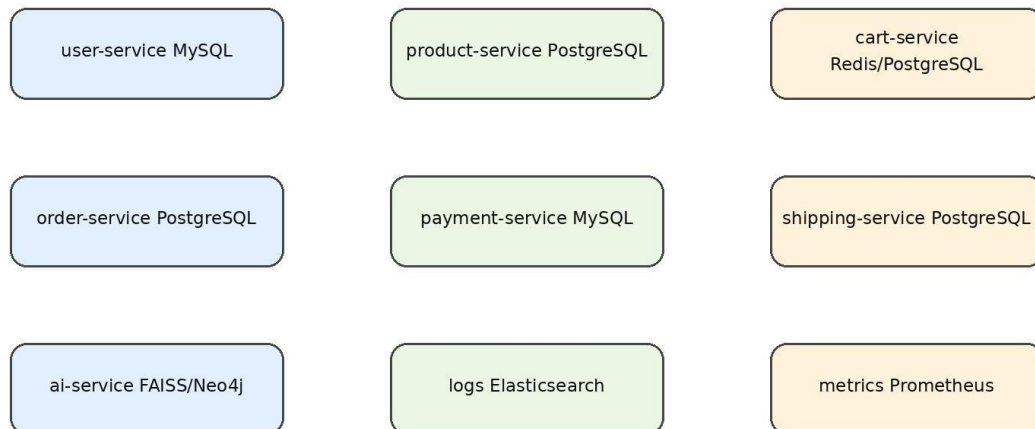
Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần đặc điểm nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Database-per-service



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

1.2.3. So sánh Monolithic vs Microservices

So sánh Monolithic vs Microservices là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần so sánh monolithic vs microservices nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

Tiêu chí	Monolithic	Microservices
Deploy	Một artifact duy nhất	Deploy từng service

Scale	Scale toàn bộ ứng dụng	Scale service cần thiết
Database	Thường dùng chung	Mỗi service sở hữu DB
Teamwork	Dễ khi team nhỏ	Phù hợp nhiều team
Debug	Dễ ở quy mô nhỏ	Cần tracing/log tập trung

1.2.4. Ưu điểm

Ưu điểm là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

1.2.5. Nhược điểm

Nhược điểm là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

1.2.6. Nguyên tắc thiết kế

Nguyên tắc thiết kế là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Ma trận giao tiếp giữa các service

	REST	Event	Shared ID	Own DB
user	login, profile			users
product	catalog		product_id	products
cart	add/remove		user_id/product_id	carts
order	checkout	OrderCreated	order_id	orders
payment	pay/status	PaymentSucceeded		payments
shipping	tracking	ShipmentCreated		shipments
ai	recommend/chatbot	BehaviorLogged		vectors/graph

Nguồn: ma trận minh họa tự xây dựng

1.3. Domain Driven Design (DDD)

Domain Driven Design (DDD) là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

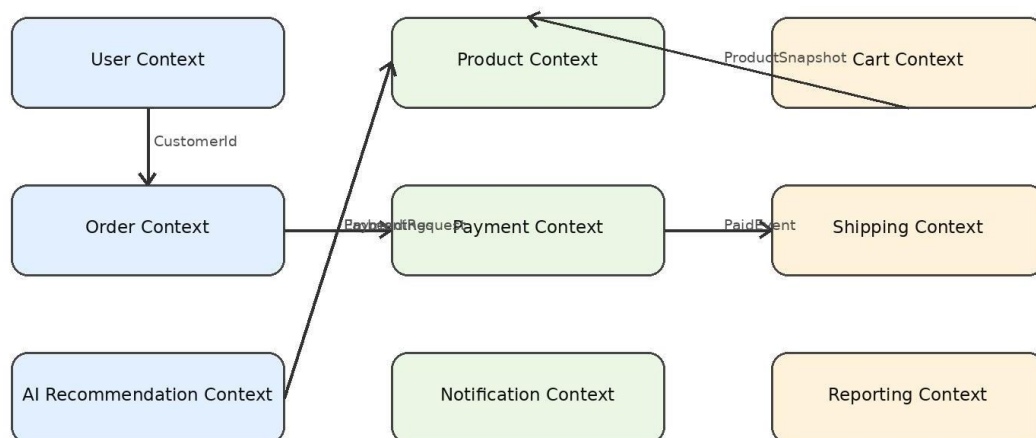
Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

Context Map theo Domain Driven Design



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

1.3.1. Mục tiêu

Mục tiêu là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

1.3.2. Các khái niệm cốt lõi

Các khái niệm cốt lõi là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần các khái niệm cốt lõi nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

1.3.3. Context Map

Context Map là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần context map nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

1.3.4. DDD trong Microservices

DDD trong Microservices là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

1.4. Case Study: Healthcare (Luyện Decomposition)

Case Study: Healthcare (Luyện Decomposition) là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary,

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

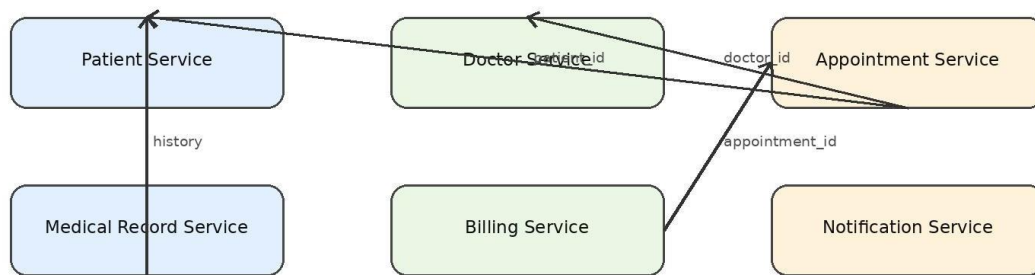
Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Case Study Healthcare: phân rã domain



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

1.4.1. Mô tả bài toán

Mô tả bài toán là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần mô tả bài toán nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

1.4.2. Bước 1: Xác định Domain

Bước 1: Xác định Domain là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, bước 1: xác định domain giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, bước 1: xác định domain yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần bước 1: xác định domain nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

1.4.3. Bước 2: Xác định Bounded Context

Bước 2: Xác định Bounded Context là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần bước 2: xác định bounded context nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

1.4.4. Bước 3: Phân rã thành Microservices

Bước 3: Phân rã thành Microservices là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, bước 3: phân rã thành microservices giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, bước 3: phân rã thành microservices yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần bước 3: phân rã thành microservices nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

1.4.5. Bước 4: Xác định quan hệ

Bước 4: Xác định quan hệ là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

1.4.6. Ví dụ API

Ví dụ API là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

```
GET /patients
```

GET /doctors

POST /appointments

```
{  
  "patient_id": 1,  
  "doctor_id": 8,  
  "time": "2026-05-20T08:00:00"  
}
```

1.5. Kết luận

Kết luận là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

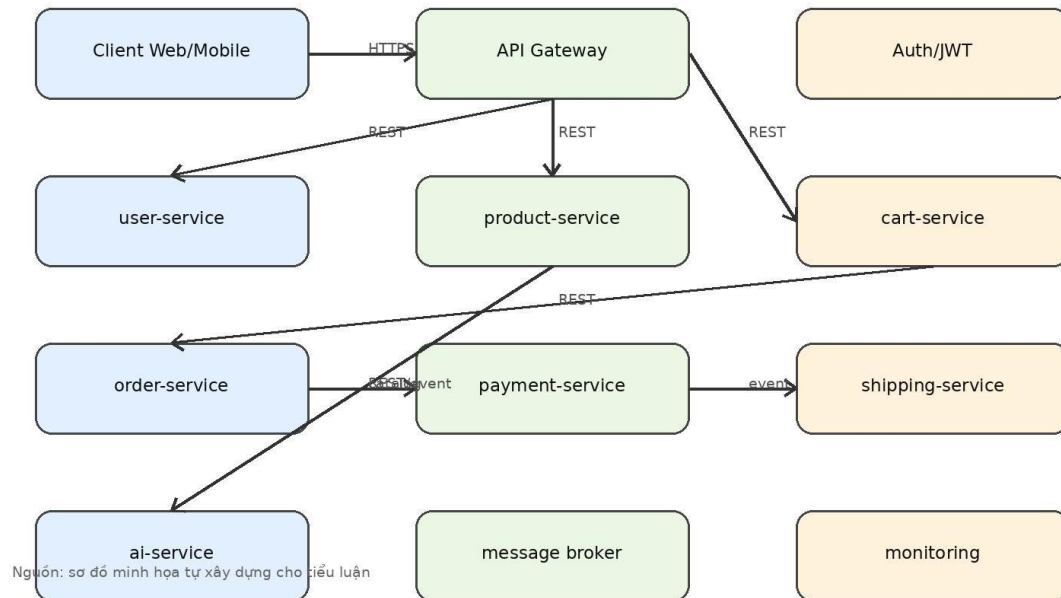
Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Chương 2. Phát triển Hệ E-Commerce Microservices

Chương 2 tập trung vào phát triển hệ e-commerce microservices. Nội dung được triển khai theo hướng từ khái niệm đến thiết kế thực hành, sau đó liên hệ trực tiếp với hệ thống ecom-final.

Tổng quan Microservices cho E-Commerce



2.1. Xác định yêu cầu

Xác định yêu cầu là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

2.1.1. Functional Requirements

Functional Requirements là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Nhóm chức năng	Mô tả	Service phụ trách
----------------	-------	-------------------

Quản lý sản phẩm	CRUD sản phẩm, danh mục, tồn kho	product-service
Người dùng	Đăng ký, đăng nhập, phân quyền	user-service
Giỏ hàng	Thêm/xóa/cập nhật sản phẩm	cart-service
Đơn hàng	Checkout, trạng thái đơn	order-service
Thanh toán	Tạo giao dịch, xác nhận thanh toán	payment-service
Giao hàng	Tạo vận đơn, tracking	shipping-service
Gợi ý	Recommendation list, chatbot	ai-service

2.1.2. Non-functional Requirements

Non-functional Requirements là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Yêu cầu phi chức năng	Mục tiêu	Cách đáp ứng
Scalability	Tăng tải từng chức năng	Scale replica theo service

Availability	Hạn chế downtime	Health check, restart, load balancing
Security	Bảo vệ API và dữ liệu	JWT, RBAC, HTTPS, validation
Maintainability	Dễ sửa đổi	Bounded context, test, API contract
Observability	Dễ giám sát	Log, metric, tracing

2.2. Phân rã hệ thống theo DDD

Phân rã hệ thống theo DDD là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

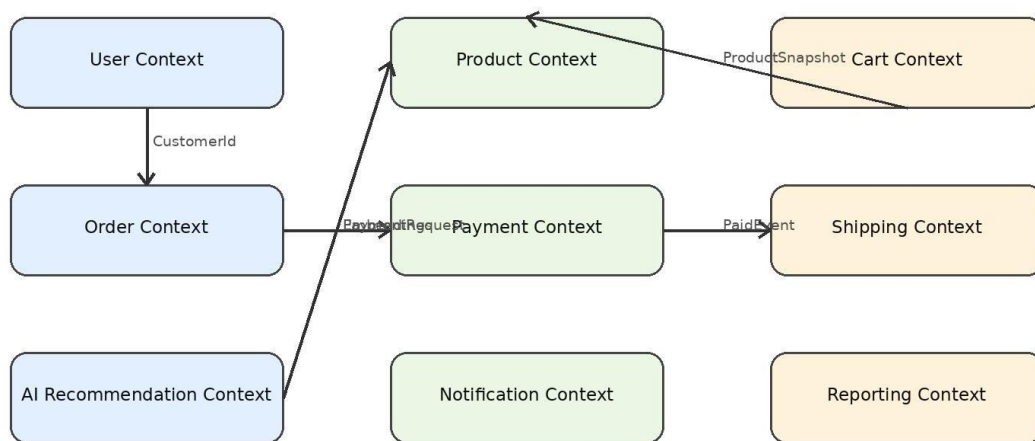
Trong tài liệu thiết kế, phần phân rã hệ thống theo ddd nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Context Map theo Domain Driven Design



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

2.2.1. Bounded Context

Bounded Context là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, bounded context giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, bounded context yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần bounded context nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

2.2.2. Nguyên tắc

Nguyên tắc là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, nguyên tắc giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, nguyên tắc yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần nguyên tắc nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

2.3. Thiết kế Product Service (Django)

Thiết kế Product Service (Django) là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

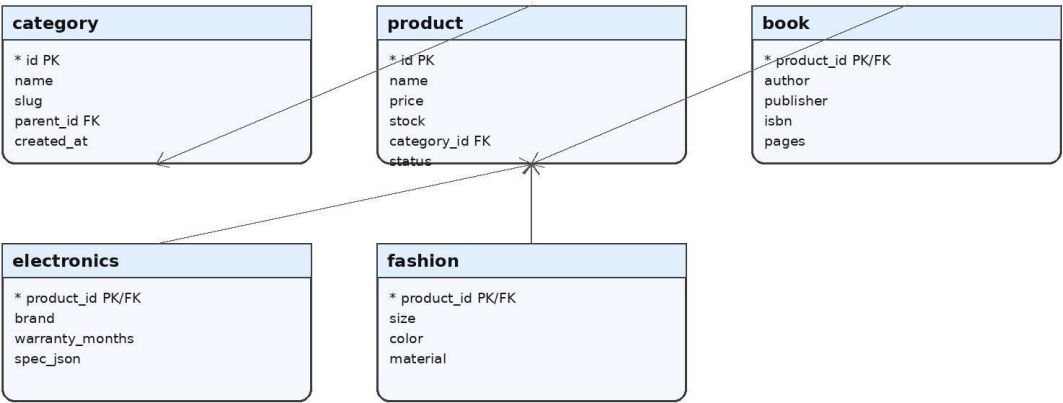
Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

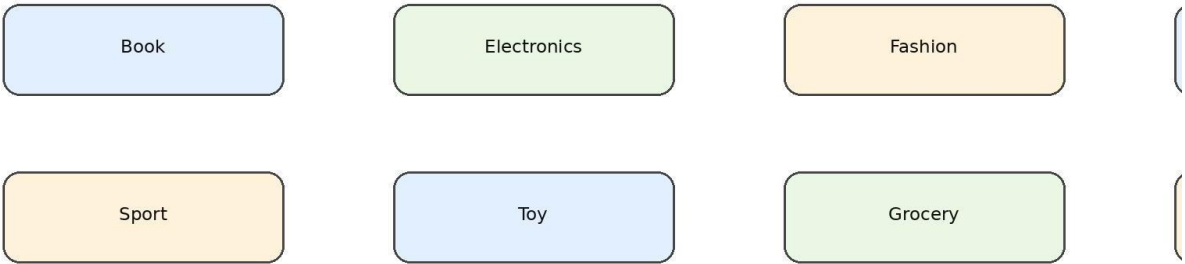
Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

ERD Product Service



Nguồn: ER diagram minh họa tự xây dựng

10 nhóm loại sản phẩm mẫu



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

2.3.1. Phân loại sản phẩm

Phân loại sản phẩm là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

2.3.2. Model tổng quát

Model tổng quát là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần model tổng quát nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

```
class Category(models.Model):
```

```
name = models.CharField(max_length=100)
slug = models.SlugField(unique=True)
```

```
class Product(models.Model):
    name = models.CharField(max_length=255)
    price = models.DecimalField(max_digits=12, decimal_places=2)
    stock = models.IntegerField(default=0)
    category = models.ForeignKey(Category, on_delete=models.CASCADE)
    status = models.CharField(max_length=20, default='active')
```

2.3.3. Chi tiết theo domain

Chi tiết theo domain là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần chi tiết theo domain nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

```
class Book(models.Model):
```

```
product = models.OneToOneField(Product, on_delete=models.CASCADE)
author = models.CharField(max_length=255)
publisher = models.CharField(max_length=255)
isbn = models.CharField(max_length=20)
```

```
class Electronics(models.Model):
```

```
    product = models.OneToOneField(Product, on_delete=models.CASCADE)
    brand = models.CharField(max_length=100)
    warranty_months = models.IntegerField()
    spec_json = models.JSONField(default=dict)
```

```
class Fashion(models.Model):
```

```
    product = models.OneToOneField(Product, on_delete=models.CASCADE)
    size = models.CharField(max_length=10)
    color = models.CharField(max_length=50)
```

2.3.4. API

API là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

```
GET /products/  
POST /products/  
GET /products/{id}/
```

PATCH /products/{id}/stock/

GET /products?category=electronics&min_price=1000000

2.4. Thiết kế User Service (Django)

Thiết kế User Service (Django) là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

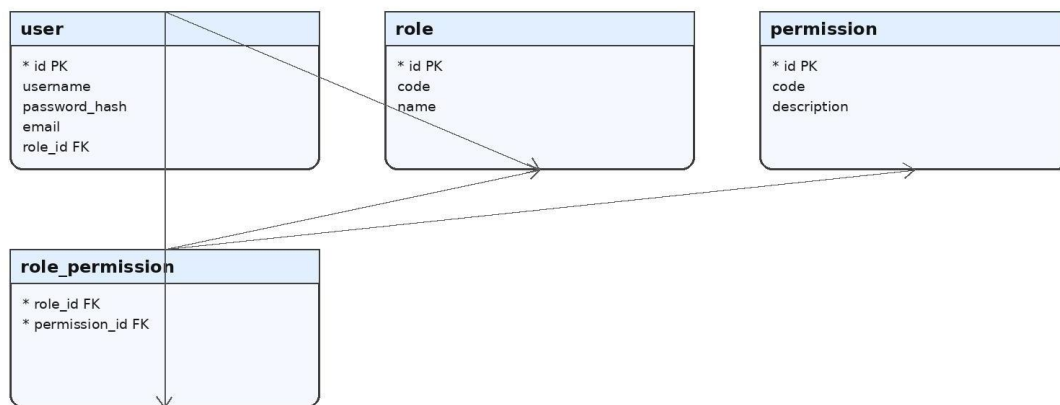
Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

ERD User Service và RBAC



Nguồn: ER diagram minh họa tự xây dựng

2.4.1. Phân loại người dùng

Phân loại người dùng là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

2.4.2. Model

Model là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần model nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

```
from django.contrib.auth.models import AbstractUser
```

```
class User(AbstractUser):
    ROLE_CHOICES = (
        ('admin', 'Admin'),
        ('staff', 'Staff'),
        ('customer', 'Customer'),
    )
    role = models.CharField(max_length=20, choices=ROLE_CHOICES,
default='customer')
    phone = models.CharField(max_length=20, blank=True)
```

2.4.3. Phân quyền (RBAC)

Phân quyền (RBAC) là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần phân quyền (rbac) nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Role	Quyền chính	Ví dụ endpoint
------	-------------	----------------

Admin	CRUD toàn bộ hệ thống	/users, /products, /orders
Staff	Xử lý đơn hàng và vận chuyển	/orders, /shipping
Customer	Mua hàng, xem lịch sử	/products, /cart, /orders/my

2.4.4. API

API là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần api nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Ảnh hưởng

POST /auth/register
POST /auth/login

POST /auth/refresh

GET /users/me

PATCH /users/me

2.5. Thiết kế Cart Service

Thiết kế Cart Service là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, thiết kế cart service giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, thiết kế cart service yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần thiết kế cart service nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

2.5.1. Model

Model là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, model giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, model yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần model nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

Code

```
class Cart(models.Model):  
    user_id = models.IntegerField()
```

```
status = models.CharField(max_length=20, default='active')

class CartItem(models.Model):
    cart = models.ForeignKey(Cart, on_delete=models.CASCADE,
related_name='items')
    product_id = models.IntegerField()
    quantity = models.PositiveIntegerField(default=1)
    price_snapshot = models.DecimalField(max_digits=12, decimal_places=2)
```

2.5.2. Logic

Logic là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, logic giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, logic yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần logic nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

2.5.3. API

API là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, api giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, api yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần api nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

API mẫu

```
POST /cart/add
GET /cart/
PATCH /cart/items/{id}
```


DELETE /cart/remove/{id}

POST /cart/clear

2.6. Thiết kế Order Service

Thiết kế Order Service là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, thiết kế order service giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, thiết kế order service yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

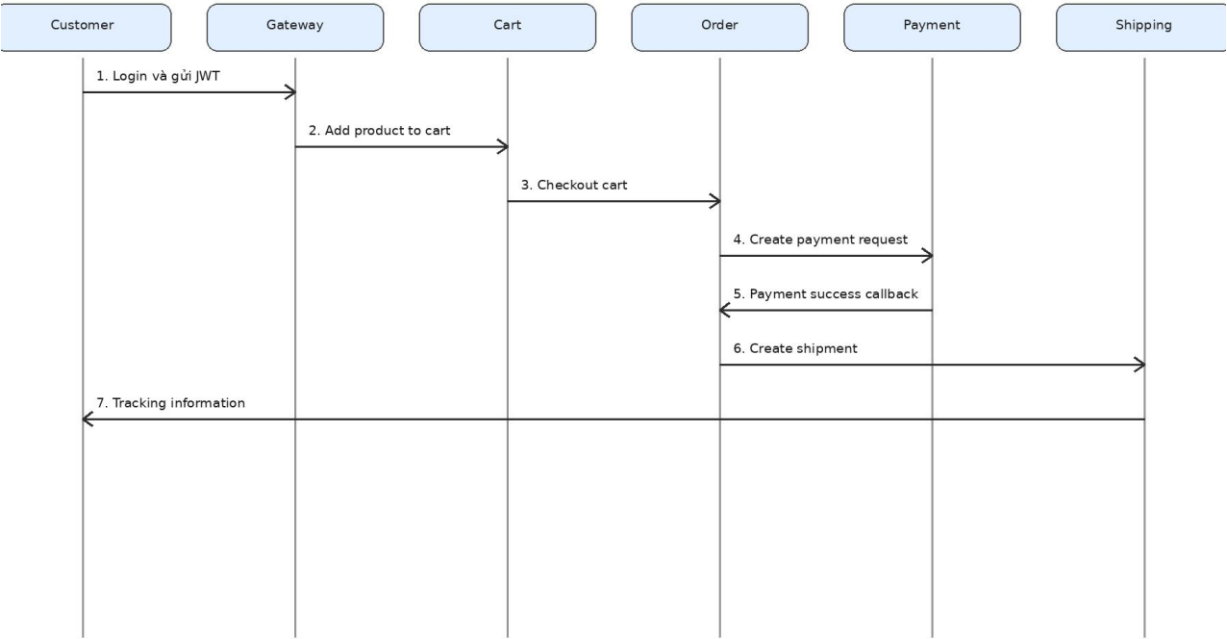
Trong tài liệu thiết kế, phần thiết kế order service nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Sequence: luồng mua hàng End-to-End



Nguồn: sequence diagram minh họa tự xây dựng

Hi vọng rằng

2.6.1. Model

Model là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, model giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, model yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần model nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

Gitnet

```
class Order(models.Model):  
    user_id = models.IntegerField()
```

```
total_price = models.DecimalField(max_digits=12, decimal_places=2)
status = models.CharField(max_length=50, default='created')

class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE,
related_name='items')
    product_id = models.IntegerField()
    quantity = models.PositiveIntegerField()
    price_snapshot = models.DecimalField(max_digits=12, decimal_places=2)
```

2.6.2. Workflow

Workflow là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, workflow giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, workflow yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần workflow nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

2.7. Thiết kế Payment Service

Thiết kế Payment Service là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, thiết kế payment service giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, thiết kế payment service yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

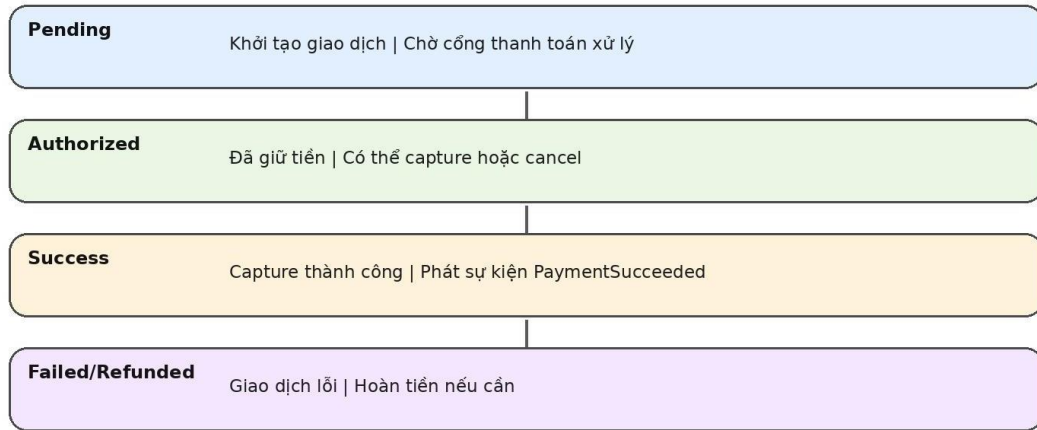
Trong tài liệu thiết kế, phần thiết kế payment service nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Payment Service: state machine



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

~~Minh Vũ Nguyễn~~

2.7.1. Model

Model là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, model giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, model yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần model nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Python

```
class Payment(models.Model):
```

```
order_id = models.IntegerField(unique=True)
amount = models.DecimalField(max_digits=12, decimal_places=2)
method = models.CharField(max_length=50)
status = models.CharField(max_length=50, default='pending')
transaction_id = models.CharField(max_length=100, blank=True)
```

2.7.2. Trạng thái

Trạng thái là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, trạng thái giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, trạng thái yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần trạng thái nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

2.7.3. API

API là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, api giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, api yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần api nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

2.8. Thiết kế Shipping Service

Thiết kế Shipping Service là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, thiết kế shipping service giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, thiết kế shipping service yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần thiết kế shipping service nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

Shipping Service: vòng đời giao hàng



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

Hình 6. Vòng đời giao hàng

2.8.1. Model

Model là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, model giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, model yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần model nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Snippet

```
class Shipment(models.Model):  
    order_id = models.IntegerField(unique=True)
```

```
address = models.TextField()
carrier = models.CharField(max_length=100, blank=True)
tracking_code = models.CharField(max_length=100, blank=True)
status = models.CharField(max_length=50, default='processing')
```


2.8.2. Trạng thái

Trạng thái là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, trạng thái giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, trạng thái yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần trạng thái nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

2.8.3. API

API là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, api giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, api yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần api nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

2.9. Luồng hệ thống tổng thể

Luồng hệ thống tổng thể là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, luồng hệ thống tổng thể giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, luồng hệ thống tổng thể yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

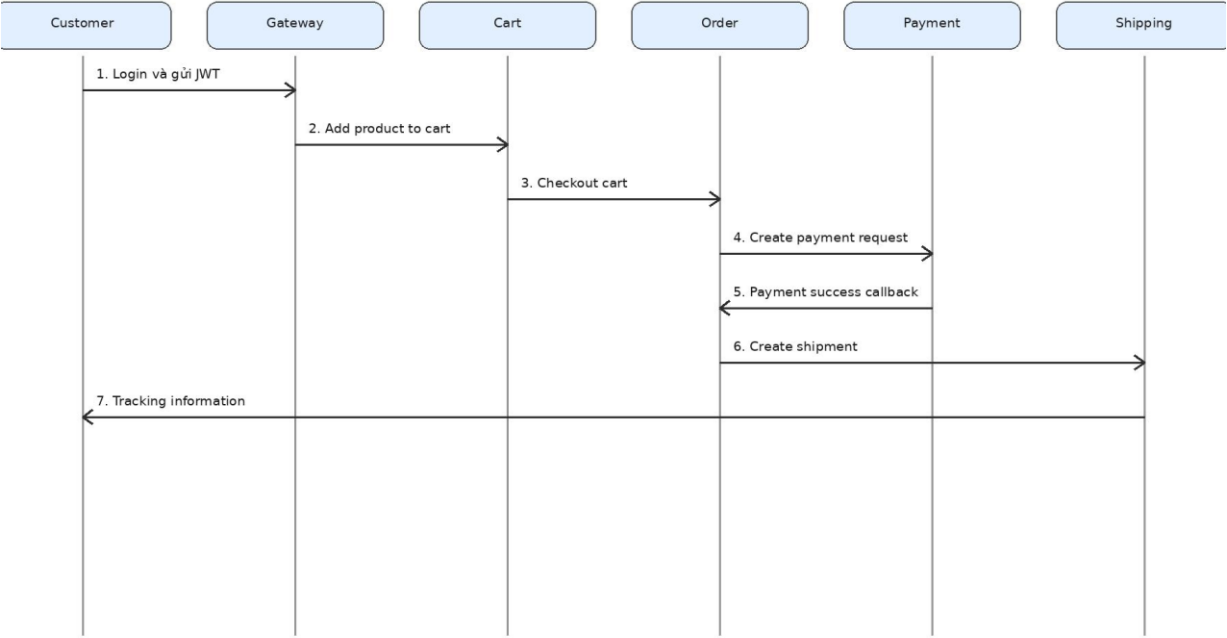
Trong tài liệu thiết kế, phần luồng hệ thống tổng thể nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Sequence: luồng mua hàng End-to-End



Nguồn: sequence diagram minh họa tự xây dựng

Hi vọng đã giúp ích

2.10. Hướng dẫn thực hành

Hướng dẫn thực hành là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, hướng dẫn thực hành giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, hướng dẫn thực hành yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần hướng dẫn thực hành nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

2.10.1. Mục tiêu

Mục tiêu là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, mục tiêu giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, mục tiêu yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần mục tiêu nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

2.10.2. Hướng dẫn vẽ Class Diagram bằng Visual Paradigm

Hướng dẫn vẽ Class Diagram bằng Visual Paradigm là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, hướng dẫn vẽ class diagram bằng visual paradigm giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, hướng dẫn vẽ class diagram bằng visual paradigm yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần hướng dẫn vẽ class diagram bằng visual paradigm nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận



2.10.3. Mapping Class Diagram sang Database

Mapping Class Diagram sang Database là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, mapping class diagram sang database giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, mapping class diagram sang database yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần mapping class diagram sang database nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

2.10.4. Thiết kế Database cho từng Service

Thiết kế Database cho từng Service là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, thiết kế database cho từng service giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, thiết kế database cho từng service yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần thiết kế database cho từng service nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

DDL mẫu

```
CREATE TABLE category (
```

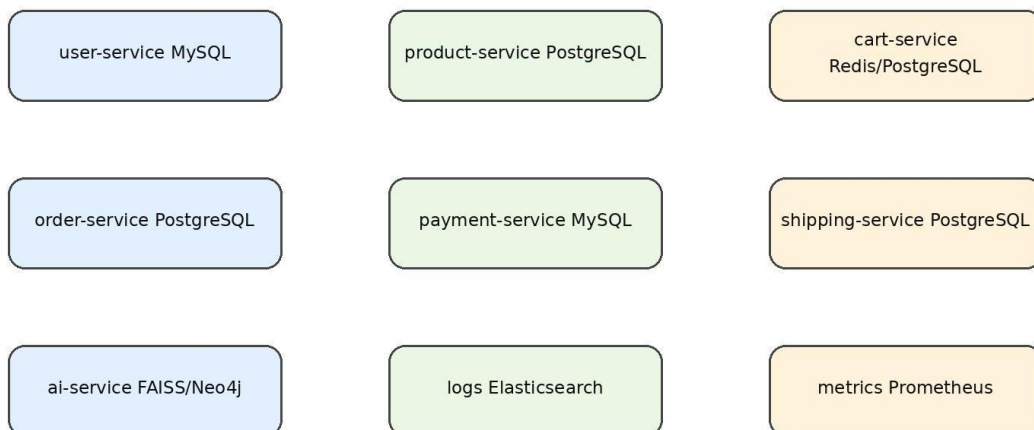
```

id SERIAL PRIMARY KEY,
name VARCHAR(100),
slug VARCHAR(100) UNIQUE
);

CREATE TABLE product (
  id SERIAL PRIMARY KEY,
  name VARCHAR(255),
  price NUMERIC(12,2),
  stock INT,
  category_id INT REFERENCES category(id)
);

```

Database-per-service



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

Thiết kế và xây dựng

2.10.5. So sánh MySQL vs PostgreSQL

So sánh MySQL vs PostgreSQL là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, so sánh mysql vs postgresql giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, so sánh mysql vs postgresql yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần so sánh mysql vs postgresql nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Bảng So sánh MySQL và PostgreSQL

Tiêu chí	MySQL	PostgreSQL
Phổ biến	Rất phổ biến	Rất phổ biến

JSON	Có hỗ trợ nhưng không mạnh bằng	Mạnh với JSON/JSONB
Quan hệ phức tạp	Tốt cho CRUD phổ thông	Mạnh cho truy vấn phức tạp
Use case	User/authentication	Product/order/schema phức tạp

2.10.6. Bài tập

Bài tập là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, bài tập giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, bài tập yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần bài tập nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

2.10.7. Checklist đánh giá

Checklist đánh giá là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, checklist đánh giá giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, checklist đánh giá yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần checklist đánh giá nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

2.11. Kết luận

Kết luận là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, kết luận giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, kết luận yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần kết luận nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

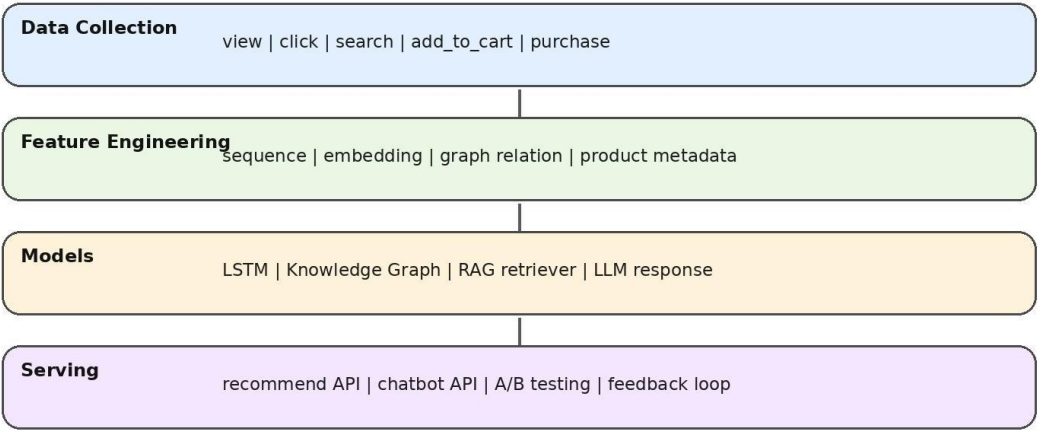
Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Chương 3. AI Service cho tư vấn sản phẩm

Chương 3 tập trung vào ai service cho tư vấn sản phẩm. Nội dung được triển khai theo hướng từ khái niệm đến thiết kế thực hành, sau đó liên hệ trực tiếp với hệ thống ecom-final.

AI Service: pipeline hybrid



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

~~HiZME AI Service~~
HiZME AI Service

3.1. Mục tiêu

Mục tiêu là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, mục tiêu giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, mục tiêu yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần mục tiêu nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

3.2. Kiến trúc AI Service

Kiến trúc AI Service là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, kiến trúc ai service giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, kiến trúc ai service yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

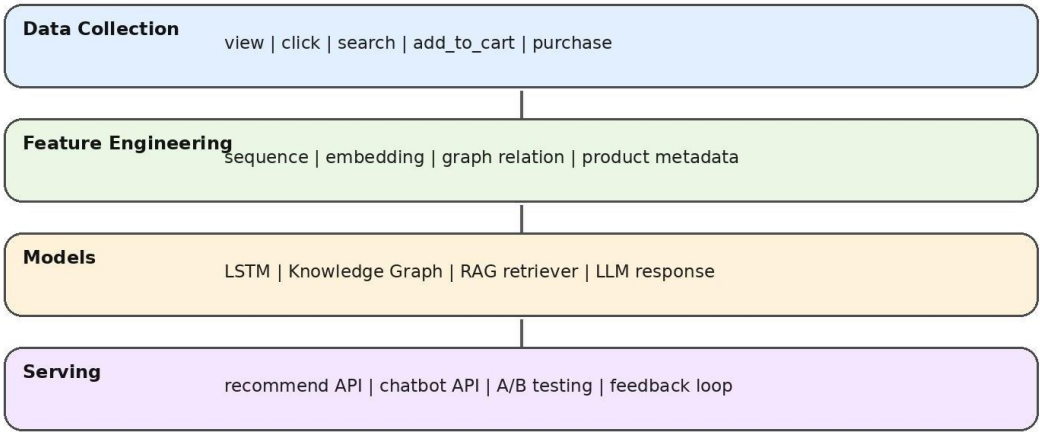
Trong tài liệu thiết kế, phần kiến trúc ai service nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

AI Service: pipeline hybrid



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

Hybrid AI: LSTM và RAG

3.3. Thu thập dữ liệu

Thu thập dữ liệu là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, thu thập dữ liệu giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, thu thập dữ liệu yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần thu thập dữ liệu nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

3.3.1. User Behavior Data

User Behavior Data là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, user behavior data giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, user behavior data yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần user behavior data nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

3.3.2. Ví dụ dataset

Ví dụ dataset là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, ví dụ dataset giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, ví dụ dataset yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần ví dụ dataset nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

Dữ liệu mẫu

```
user_id,product_id,action,time  
1,101,view,2026-04-01T08:00:00
```

1,102,add_to_cart,2026-04-01T08:03:00
2,205,search,2026-04-01T09:10:00
2,207,purchase,2026-04-01T09:20:00

3.4. Mô hình LSTM (Sequence Modeling)

Mô hình LSTM (Sequence Modeling) là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, mô hình lstm (sequence modeling) giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, mô hình lstm (sequence modeling) yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

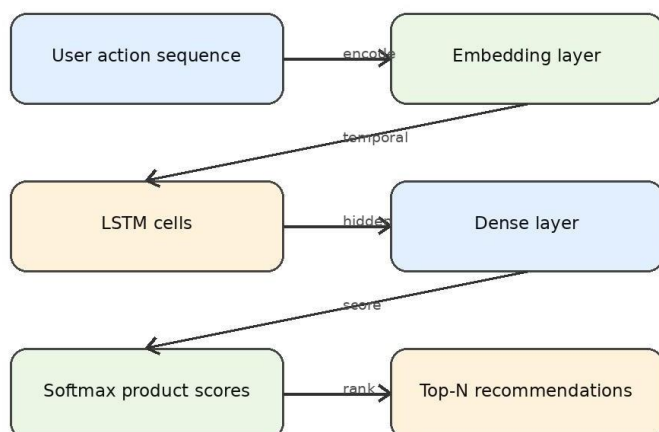
Trong tài liệu thiết kế, phần mô hình lstm (sequence modeling) nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

LSTM Recommendation Model



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

~~Hi 2Mô hình LSTM để sản phẩm~~

3.4.1. Ý tưởng

Ý tưởng là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, ý tưởng giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, ý tưởng yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần ý tưởng nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

3.4.2. Model chi tiết

Model chi tiết là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, model chi tiết giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, model chi tiết yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần model chi tiết nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Python

```
import torch
```

```
import torch.nn as nn

class LSTMModel(nn.Module):
    def __init__(self, input_dim=64, hidden_dim=128, output_dim=1000):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        out, _ = self.lstm(x)
        out = out[:, -1, :]
        return self.fc(out)
```


3.4.3. Training

Training là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, training giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, training yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần training nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Tập mẫu

```
critterion = nn.CrossEntropyLoss()
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

```
for epoch in range(epochs):
```

```
    optimizer.zero_grad()
```

```
    output = model(x_train)
```

```
    loss = criterion(output, y_train)
```

```
    loss.backward()
```

```
    optimizer.step()
```

3.5. Knowledge Graph với Neo4j

Knowledge Graph với Neo4j là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, knowledge graph với neo4j giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, knowledge graph với neo4j yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

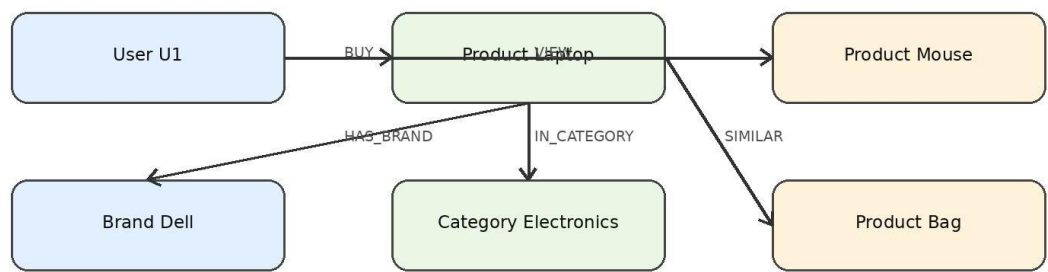
Trong tài liệu thiết kế, phần knowledge graph với neo4j nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Knowledge Graph với Neo4j



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

~~Không ghi rõ nguồn~~

3.5.1. Mô hình đồ thị

Mô hình đồ thị là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, mô hình đồ thị giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, mô hình đồ thị yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần mô hình đồ thị nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

3.5.2. Ví dụ Cypher

Ví dụ Cypher là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, ví dụ cypher giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, ví dụ cypher yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần ví dụ cypher nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

Graph

```
CREATE (u:User {id: 1})
```

```
CREATE (p:Product {id: 101, name: 'Laptop A'})  
CREATE (c:Category {name: 'Electronics'})  
CREATE (u)-[:BUY]->(p)  
CREATE (p)-[:IN_CATEGORY]->(c)
```

3.5.3. Truy vấn gợi ý

Truy vấn gợi ý là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, truy vấn gợi ý giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, truy vấn gợi ý yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần truy vấn gợi ý nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Gợi ý

--


```
MATCH (u:User {id: 1})-[:BUY]->(p)-[:SIMILAR]->(rec)  
RETURN rec  
ORDER BY rec.score DESC  
LIMIT 10
```

3.6. RAG (Retrieval-Augmented Generation)

RAG (Retrieval-Augmented Generation) là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, rag (retrieval-augmented generation) giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, rag (retrieval-augmented generation) yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

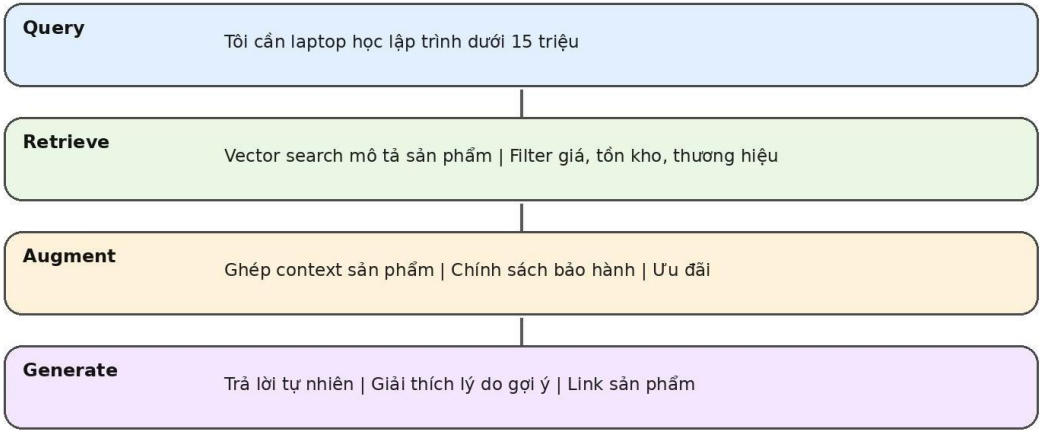
Trong tài liệu thiết kế, phần rag (retrieval-augmented generation) nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

RAG chatbot tư vấn sản phẩm



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

~~Minh họa RAG chatbot tư vấn sản phẩm~~

3.6.1. Pipeline

Pipeline là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, pipeline giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, pipeline yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần pipeline nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

3.6.2. Vector Database

Vector Database là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, vector database giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, vector database yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần vector database nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

3.6.3. Ví dụ

Ví dụ là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, ví dụ giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, ví dụ yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần ví dụ nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Request

```
query = 'laptop gaming dưới 20 triệu'  
embedding = embedder.encode(query)  
results = vector_db.search(embedding, top_k=5)
```

```
context = build_context(results)
response = llm.generate(prompt=query, context=context)
```

3.7. Kết hợp Hybrid Model

Kết hợp Hybrid Model là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, kết hợp hybrid model giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, kết hợp hybrid model yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần kết hợp hybrid model nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

Hy

```
final_score = 0.45 * lstm_score + 0.30 * graph_score + 0.25 * rag_score  
recommendations = rank_by(final_score)[:10]
```

3.8. Hai dạng AI Service

Hai dạng AI Service là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, hai dạng ai service giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, hai dạng ai service yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần hai dạng ai service nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

3.8.1. Recommendation List

Recommendation List là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, recommendation list giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, recommendation list yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần recommendation list nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Recommendation

--

GET /recommend?user_id=1

Response:

```
[  
  {"product_id": 101, "score": 0.92, "reason": "similar to purchased laptop"},  
  {"product_id": 205, "score": 0.87, "reason": "frequently bought together"}  
]
```

3.8.2. Chatbot tư vấn

Chatbot tư vấn là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, chatbot tư vấn giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, chatbot tư vấn yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần chatbot tư vấn nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Chat

POST /chatbot

```
{  
  "user_id": 1,  
  "message": "Tôi cần laptop học lập trình dưới 15 triệu"  
}
```

Response:

```
{  
  "answer": "Bạn có thể tham khảo Laptop X...",  
  "products": [101, 205]  
}
```

3.9. Triển khai AI Service

Triển khai AI Service là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, triển khai ai service giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, triển khai ai service yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần triển khai ai service nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

3.9.1. Tech stack

Tech stack là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, tech stack giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, tech stack yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần tech stack nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

Fig 5.14.4

Thành phần	Công nghệ đề xuất	Vai trò
API	FastAPI	Serving recommendation/chatbot

Sequence model	PyTorch hoặc TensorFlow	LSTM dự đoán hành vi
Graph	Neo4j	Quan hệ user-product
Vector DB	FAISS hoặc ChromaDB	Tìm kiếm ngữ nghĩa
Cache	Redis	Cache top-N recommendation

3.9.2. Kiến trúc

Kiến trúc là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, kiến trúc giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, kiến trúc yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần kiến trúc nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

3.10. Bài tập

Bài tập là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, bài tập giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, bài tập yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần bài tập nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

3.11. Checklist đánh giá

Checklist đánh giá là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, checklist đánh giá giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, checklist đánh giá yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần checklist đánh giá nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

3.12. Kết luận

Kết luận là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, kết luận giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, kết luận yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần kết luận nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

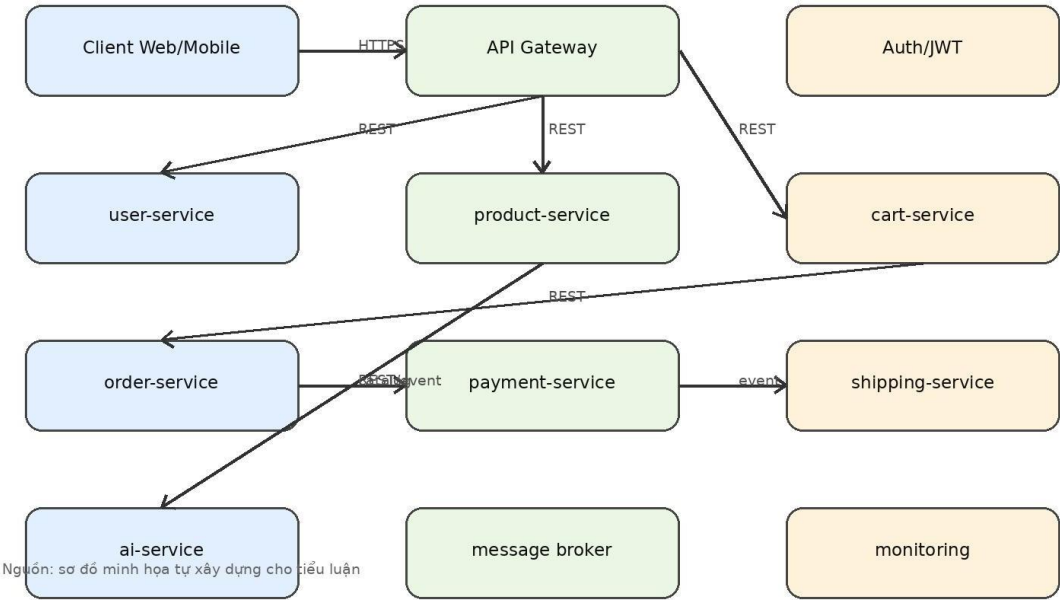
Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Chương 4. Xây dựng hệ thống hoàn chỉnh

Chương 4 tập trung vào xây dựng hệ thống hoàn chỉnh. Nội dung được triển khai theo hướng từ khái niệm đến thiết kế thực hành, sau đó liên hệ trực tiếp với hệ thống ecom-final.

Tổng quan Microservices cho E-Commerce



~~Hệ thống hoàn chỉnh~~

4.1. Kiến trúc tổng thể

Kiến trúc tổng thể là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, kiến trúc tổng thể giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, kiến trúc tổng thể yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

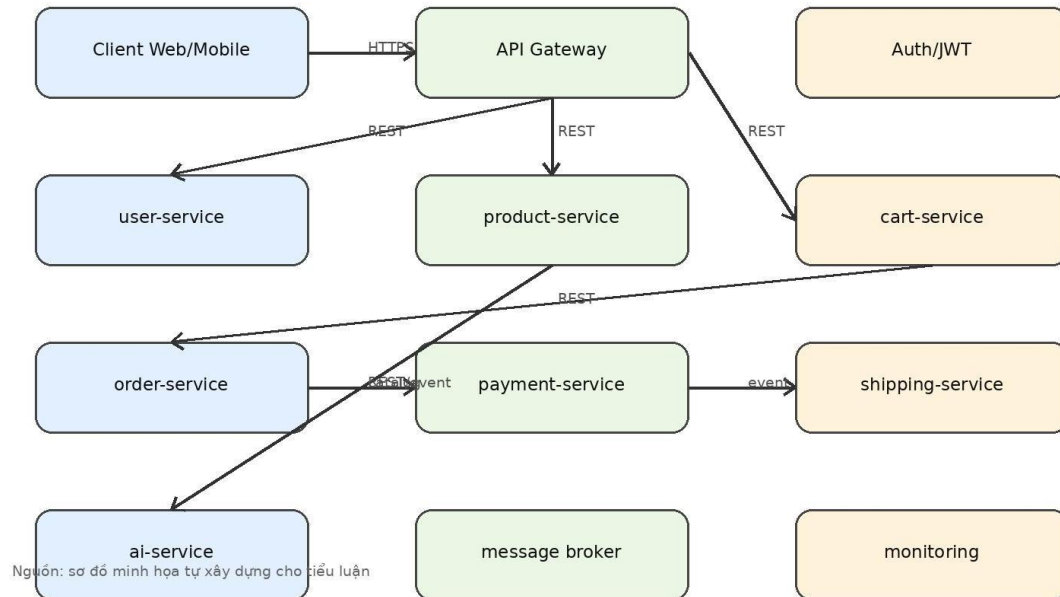
Trong tài liệu thiết kế, phần kiến trúc tổng thể nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Tổng quan Microservices cho E-Commerce



Minh họa

4.1.1. Mô hình hệ thống

Mô hình hệ thống là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, mô hình hệ thống giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, mô hình hệ thống yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần mô hình hệ thống nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

4.1.2. Nguyên tắc

Nguyên tắc là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, nguyên tắc giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, nguyên tắc yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần nguyên tắc nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

4.2. System Architecture

System Architecture là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, system architecture giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, system architecture yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần system architecture nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

4.2.1. Overview

Overview là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, overview giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, overview yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần overview nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

4.2.2. Microservice Architecture

Microservice Architecture là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, microservice architecture giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, microservice architecture yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

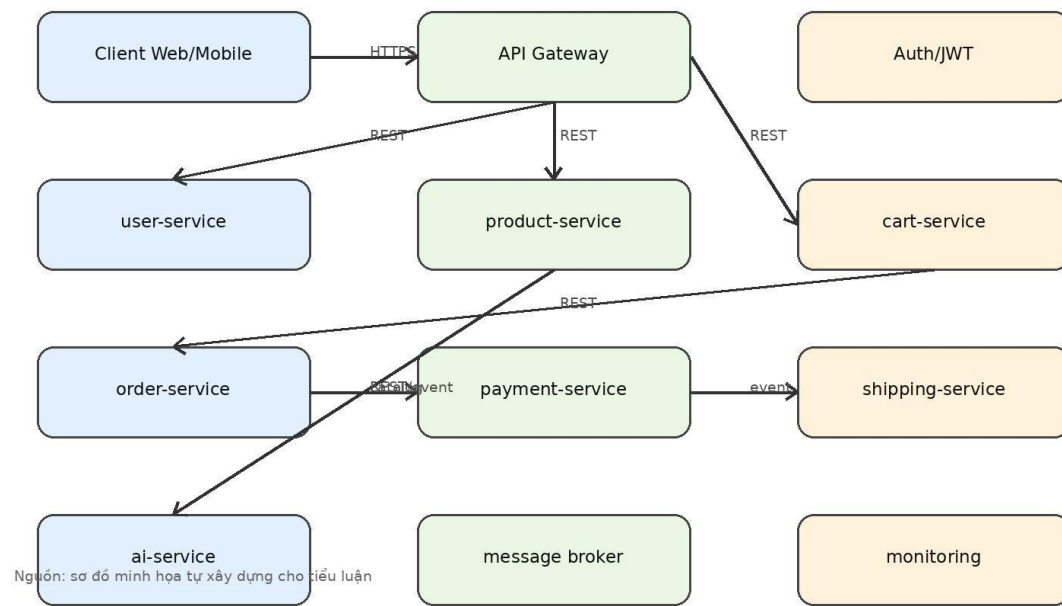
Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần microservice architecture nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Tổng quan Microservices cho E-Commerce



Minh họa

4.2.3. API Gateway

API Gateway là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, api gateway giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, api gateway yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần api gateway nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

4.2.4. Service Communication

Service Communication là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, service communication giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, service communication yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần service communication nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

4.2.5. Containerization and Deployment

Containerization and Deployment là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, containerization and deployment giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, containerization and deployment yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần containerization and deployment nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

4.2.6. System Structure

System Structure là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, system structure giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, system structure yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần system structure nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Summary



```
ecom-final/  
|-- gateway/  
| |-- nginx.conf  
|-- user-service/  
|-- product-service/  
|-- cart-service/  
|-- order-service/  
|-- payment-service/  
|-- shipping-service/  
|-- ai-service/  
|-- infrastructure/  
| |-- docker-compose.yml
```

4.2.7. Design Principles

Design Principles là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, design principles giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, design principles yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần design principles nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

4.2.8. Security Considerations

Security Considerations là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, security considerations giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, security considerations yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

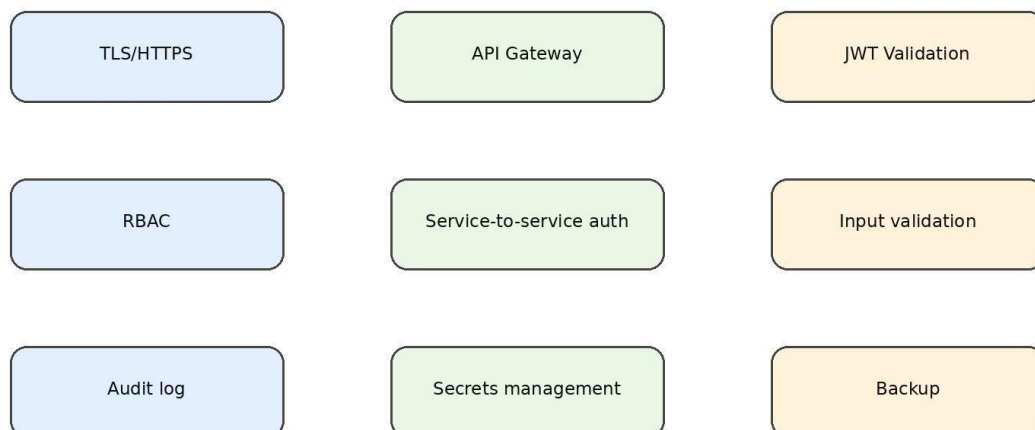
Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần security considerations nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Các lớp bảo mật trong hệ thống



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

Thư viện tài liệu

4.2.9. Discussion

Discussion là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, discussion giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, discussion yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần discussion nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

4.3. API Gateway (Nginx)

API Gateway (Nginx) là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, api gateway (nginx) giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, api gateway (nginx) yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

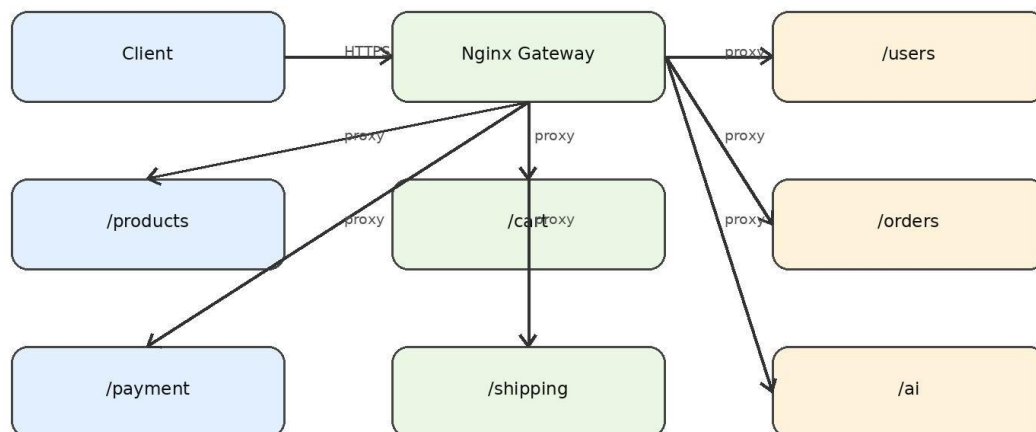
Trong tài liệu thiết kế, phần api gateway (nginx) nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

API Gateway bằng Nginx



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

H2N Công Nghệ

4.3.1. Vai trò

Vai trò là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, vai trò giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, vai trò yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần vai trò nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

4.3.2. Cấu hình mẫu

Cấu hình mẫu là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, cấu hình mẫu giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, cấu hình mẫu yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần cấu hình mẫu nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

Nội dung

```
server {  
  listen 80;
```

```
location /users/ {  
    proxy_pass http://user-service:8000;  
}  
location /products/ {  
    proxy_pass http://product-service:8001;  
}  
location /orders/ {  
    proxy_pass http://order-service:8003;  
}  
}
```


4.4. Authentication (JWT)

Authentication (JWT) là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, authentication (jwt) giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, authentication (jwt) yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

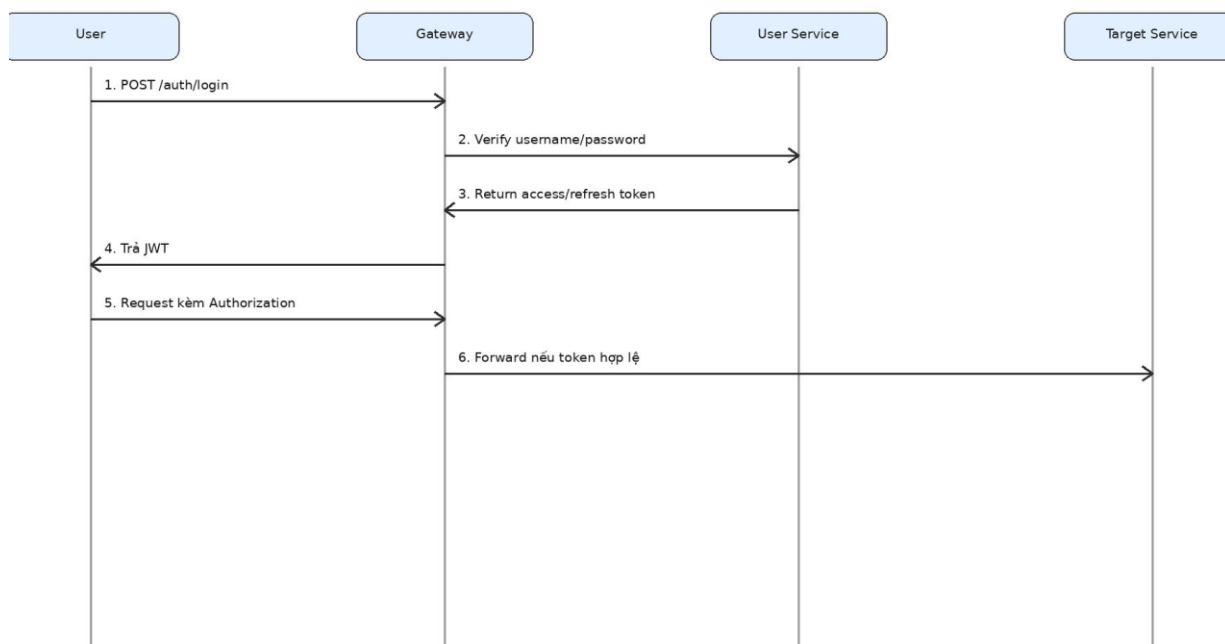
Trong tài liệu thiết kế, phần authentication (jwt) nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

Luồng xác thực JWT



Nguồn: sequence diagram minh họa tự xây dựng

Nguyễn Thị Ngọc Minh

4.4.1. Cài đặt

Cài đặt là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, cài đặt giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, cài đặt yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần cài đặt nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

JWT

```
pip install django-rest-framework-simplejwt
```

4.4.2. Cấu hình

Cấu hình là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, cấu hình giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, cấu hình yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần cấu hình nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Cấu hình

```
REST_FRAMEWORK = {  
    'DEFAULT_AUTHENTICATION_CLASSES': (  

```

```
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    )
}

from rest_framework_simplejwt.views import TokenObtainPairView,
TokenRefreshView
```

4.4.3. Luồng

Luồng là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, luồng giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, luồng yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần luồng nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

4.5. Giao tiếp giữa các Service

Giao tiếp giữa các Service là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, giao tiếp giữa các service giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, giao tiếp giữa các service yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần giao tiếp giữa các service nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

4.5.1. REST API call

REST API call là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, rest api call giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, rest api call yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần rest api call nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

REST API

```
import requests
```

```
response = requests.get(
    'http://product-service:8001/products/101',
    timeout=3
)
response.raise_for_status()
product = response.json()
```

4.5.2. Best Practice

Best Practice là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, best practice giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, best practice yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần best practice nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

4.6. Docker hóa hệ thống

Docker hóa hệ thống là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, docker hóa hệ thống giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, docker hóa hệ thống yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

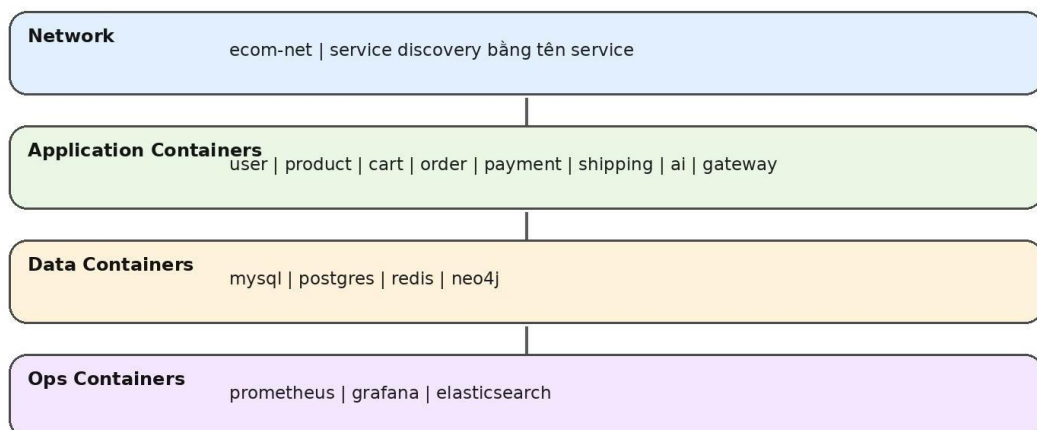
Trong tài liệu thiết kế, phần docker hóa hệ thống nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Docker Compose cho môi trường phát triển



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

~~Đề tài: Docker Compose~~

4.6.1. Dockerfile (Django)

Dockerfile (Django) là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, dockerfile giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, dockerfile yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần dockerfile nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

Dockerfile

```
FROM python:3.10-slim
WORKDIR /app
```

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]

4.6.2. docker-compose.yml

docker-compose.yml là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, docker-compose.yml giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, docker-compose.yml yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần docker-compose.yml nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

đơn giản

```
version: '3.9'
```



```
services:
  user-service:
    build: ./user-service
    ports:
      - '8000:8000'
  product-service:
    build: ./product-service
    ports:
      - '8001:8000'
  gateway:
    build: ./gateway
    ports:
      - '80:80'
```

4.7. Luồng hệ thống (End-to-End)

Luồng hệ thống (End-to-End) là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, luồng hệ thống (end-to-end) giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, luồng hệ thống (end-to-end) yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

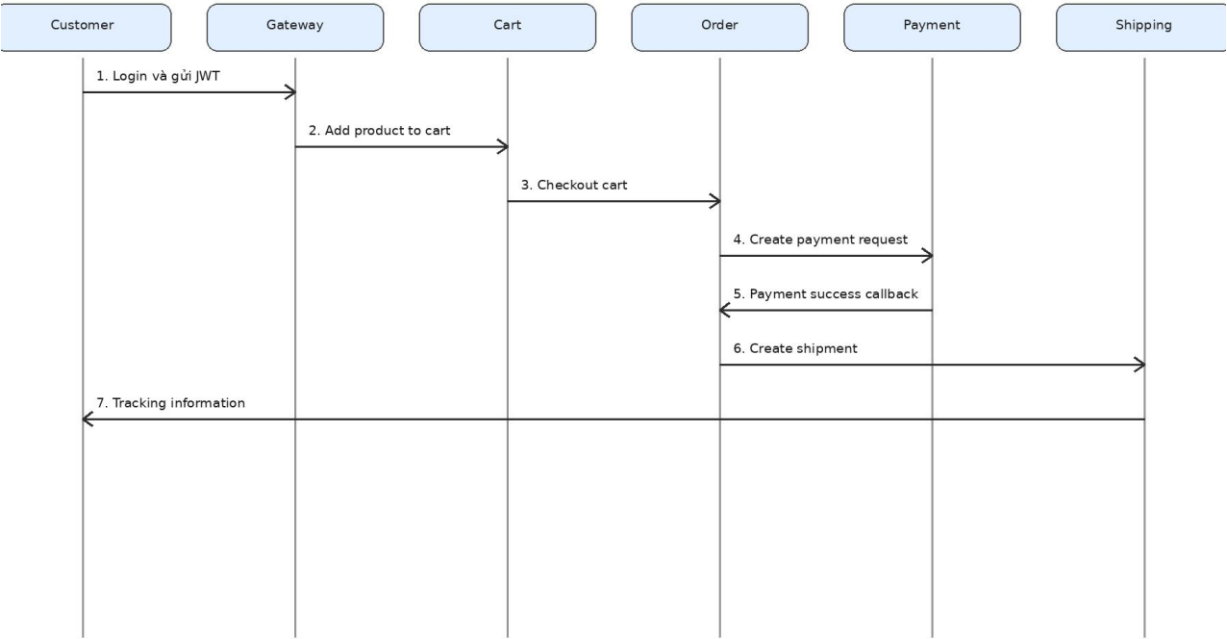
Trong tài liệu thiết kế, phần luồng hệ thống (end-to-end) nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

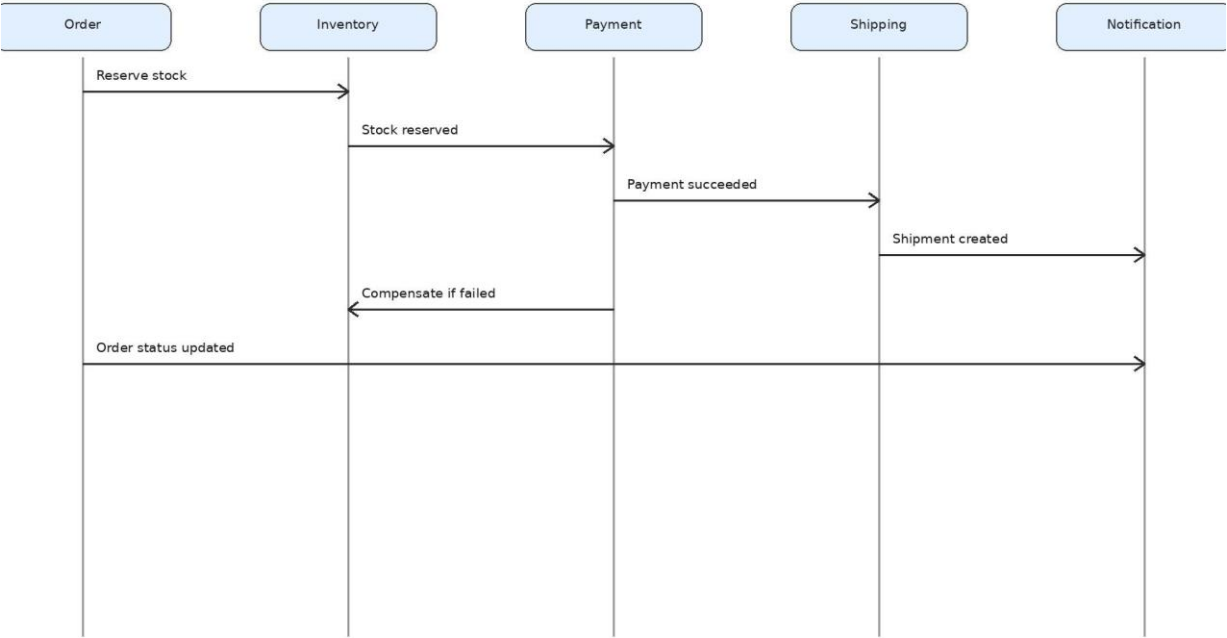
Sequence: luồng mua hàng End-to-End



Nguồn: sequence diagram minh họa tự xây dựng

HL3 End-to-End mua hàng

Saga Pattern cho checkout



Nguồn: sequence diagram minh họa tự xây dựng

HL3 Saga Pattern checkout

4.7.1. Use case: Mua hàng

Use case: Mua hàng là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, use case: mua hàng giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, use case: mua hàng yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần use case: mua hàng nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

4.7.2. Sequence logic

Sequence logic là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, sequence logic giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, sequence logic yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần sequence logic nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

4.8. Triển khai Kubernetes (Optional)

Triển khai Kubernetes (Optional) là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, triển khai kubernetes giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, triển khai kubernetes yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

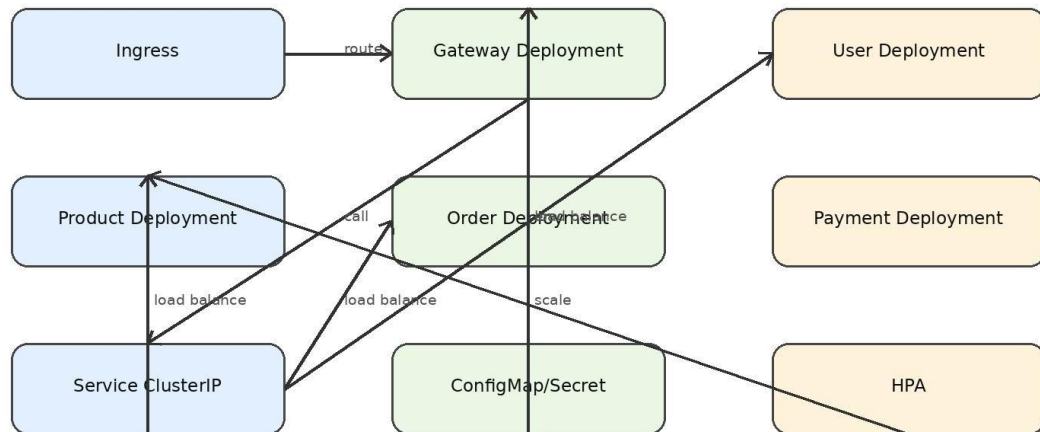
Trong tài liệu thiết kế, phần triển khai kubernetes nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

Triển khai Kubernetes tùy chọn



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

4.8.1. Deployment

Deployment là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

```
apiVersion: apps/v1
kind: Deployment
```



```
metadata:
  name: user-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: user-service
  template:
    metadata:
      labels:
        app: user-service
    spec:
      containers:
        - name: user-service
          image: ecom/user-service:latest
```

4.8.2. Service

Service là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

apiVersion: v1

```
kind: Service
metadata:
  name: user-service
spec:
  type: ClusterIP
  selector:
    app: user-service
  ports:
    - port: 8000
      targetPort: 8000
```

4.9. Logging và Monitoring

Logging và Monitoring là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

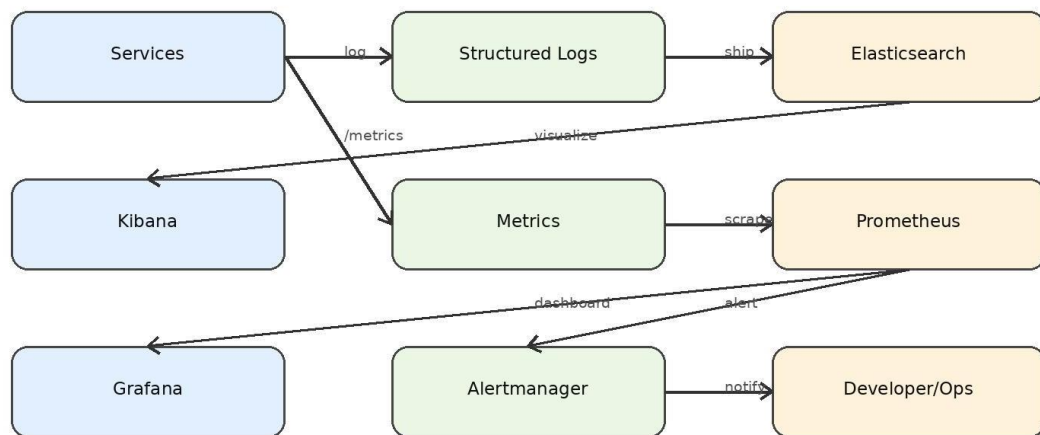
Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Logging và Monitoring



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

4.10. Đánh giá hệ thống

Đánh giá hệ thống là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

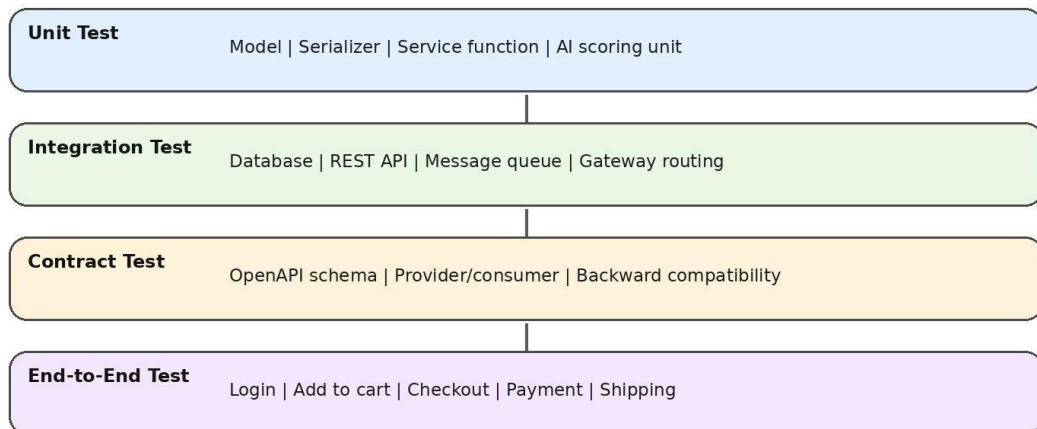
Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Trong tài liệu thiết kế, phần đánh giá hệ thống nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

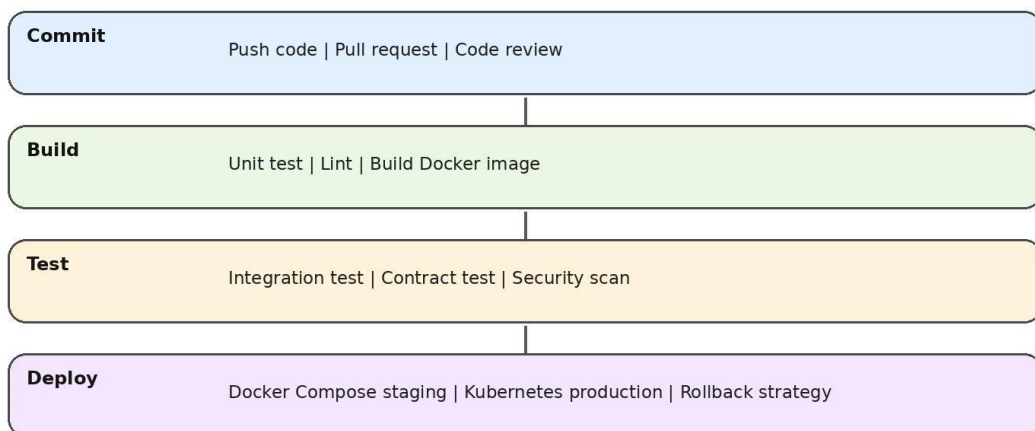
Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Kim tự tháp kiểm thử



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

CI/CD pipeline đề xuất



Nguồn: sơ đồ minh họa tự xây dựng cho tiểu luận

4.10.1. Hiệu năng

Hiệu năng là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần hiệu năng nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

Metric	Ý nghĩa	Mục tiêu mẫu
p95 latency	95% request nhanh hơn ngưỡng	< 500ms cho API đọc

Throughput	Số request/giây	Tăng tuyến tính khi scale
Error rate	Tỷ lệ lỗi HTTP 5xx	< 1%
CPU/Memory	Tài nguyên container	Không vượt ngưỡng cảnh báo

4.10.2. Khả năng mở rộng

Khả năng mở rộng là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần khả năng mở rộng nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

4.10.3. Ưu điểm

Ưu điểm là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần ưu điểm nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

4.10.4. Nhược điểm

Nhược điểm là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần nhược điểm nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

4.11. Bài tập thực hành

Bài tập thực hành là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Trong tài liệu thiết kế, phần bài tập thực hành nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ ba là không kiểm soát version API. Khi một service thay đổi response nhưng service gọi chưa cập nhật, hệ thống có thể lỗi dù từng service riêng lẻ vẫn chạy.

4.12. Checklist đánh giá

Checklist đánh giá là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

5. Kết luận chung

Tiểu luận đã trình bày quá trình thiết kế hệ thống e-commerce từ kiến trúc monolithic truyền thống đến microservices hiện đại. Điểm then chốt là không chia hệ thống theo lớp kỹ thuật mà chia theo domain, bounded context và năng lực nghiệp vụ.

DDD giúp xác định ranh giới rõ ràng giữa User, Product, Cart, Order, Payment, Shipping và AI Recommendation. Mỗi context được hiện thực hóa thành một service độc lập, có database riêng, API riêng và quy tắc nghiệp vụ riêng.

Phần AI Service mở rộng giá trị của hệ thống bằng khả năng cá nhân hóa. Sự kết hợp giữa LSTM, Knowledge Graph và RAG tạo ra phương pháp hybrid vừa khai thác hành vi người dùng, vừa hiểu quan hệ sản phẩm, vừa trả lời được truy vấn tự nhiên.

Hệ thống ecom-final có thể được triển khai bằng Docker Compose cho môi trường phát triển và mở rộng sang Kubernetes cho production. Để vận hành ổn định, hệ thống cần API Gateway, JWT, RBAC, logging, monitoring, health check và chiến lược kiểm thử đầy đủ.

Kết quả cuối cùng là một bản thiết kế có tính học thuật và thực hành, phù hợp yêu cầu của môn học Kiến trúc và Thiết kế Phần mềm.

- Microservices phù hợp hệ thống lớn và thay đổi thường xuyên.
- DDD giúp mô hình hóa đúng domain và giảm coupling.
- AI nâng cao trải nghiệm mua sắm bằng gợi ý và chatbot.
- Docker, Kubernetes và monitoring là nền tảng vận hành cần thiết.

Phụ lục A. API Specification

Phụ lục A. API Specification cung cấp phần minh họa thực hành để tài liệu có thể dùng trực tiếp khi triển khai hoặc thuyết trình. Các nội dung trong phụ lục bổ sung cho phần thiết kế chính bằng endpoint, schema, test case và hướng dẫn vận hành.

User Service

Mục User Service mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

```
POST /auth/register
POST /auth/login
POST /auth/refresh
GET /users/me
PATCH /users/me
GET /users/{id}
```

Product Service

Mục Product Service mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

```
GET /products
POST /products
GET /products/{id}
PATCH /products/{id}
DELETE /products/{id}
GET /categories
```

Cart Service

Mục Cart Service mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

```
GET /cart
POST /cart/add
PATCH /cart/items/{id}
DELETE /cart/items/{id}
POST /cart/checkout
```

Order Service

Mục Order Service mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

```
POST /orders
GET /orders/{id}
GET /orders/my
PATCH /orders/{id}/status
POST /orders/{id}/cancel
```

Payment Service

Mục Payment Service mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

```
POST /payment/pay  
GET /payment/status/{order_id}  
POST /payment/webhook  
POST /payment/refund
```

Shipping Service

Mục Shipping Service mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

```
POST /shipping/create  
GET /shipping/status/{order_id}  
PATCH /shipping/{id}/tracking
```

AI Service

Mục AI Service mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

```
GET /recommend?user_id=1
POST /chatbot
POST /behavior/log
POST /model/retrain
```


Phụ lục B. Database DDL

Phụ lục B. Database DDL cung cấp phần minh họa thực hành để tài liệu có thể dùng trực tiếp khi triển khai hoặc thuyết trình. Các nội dung trong phụ lục bổ sung cho phần thiết kế chính bằng endpoint, schema, test case và hướng dẫn vận hành.

User DB

Mục User DB mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

```
CREATE TABLE user_account (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(100) UNIQUE,  
  password_hash VARCHAR(255),  
  email VARCHAR(255),  
  role VARCHAR(20),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Product DB

Mục Product DB mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

```
CREATE TABLE category (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100),  
  slug VARCHAR(100) UNIQUE  
);  
CREATE TABLE product (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(255),  
  price NUMERIC(12,2),  
  stock INT,  
  category_id INT REFERENCES category(id),  
  status VARCHAR(20)  
);
```

Cart DB

Mục Cart DB mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

```
CREATE TABLE cart (  
  id SERIAL PRIMARY KEY,  
  user_id INT,  
  status VARCHAR(20)  
);  
CREATE TABLE cart_item (  
  id SERIAL PRIMARY KEY,  
  cart_id INT REFERENCES cart(id),  
  product_id INT,  
  quantity INT,  
  price_snapshot NUMERIC(12,2)  
);
```

Order DB

Mục Order DB mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

```
CREATE TABLE orders (  
  id SERIAL PRIMARY KEY,  
  user_id INT,  
  total_price NUMERIC(12,2),  
  status VARCHAR(50),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
CREATE TABLE order_item (  
  id SERIAL PRIMARY KEY,  
  order_id INT REFERENCES orders(id),  
  product_id INT,  
  quantity INT,  
  price_snapshot NUMERIC(12,2)  
);
```

Payment DB

Mục Payment DB mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

```
CREATE TABLE payment (  
  id SERIAL PRIMARY KEY,  
  order_id INT UNIQUE,  
  amount NUMERIC(12,2),  
  method VARCHAR(50),  
  status VARCHAR(50),  
  transaction_id VARCHAR(100)  
);
```

Shipping DB

Mục Shipping DB mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

```
CREATE TABLE shipment (  
  id SERIAL PRIMARY KEY,  
  order_id INT UNIQUE,  
  address TEXT,  
  carrier VARCHAR(100),  
  tracking_code VARCHAR(100),  
  status VARCHAR(50)  
);
```

Phụ lục C. Test cases và checklist nghiệm thu

Phụ lục C. Test cases và checklist nghiệm thu cung cấp phần minh họa thực hành để tài liệu có thể dùng trực tiếp khi triển khai hoặc thuyết trình. Các nội dung trong phụ lục bổ sung cho phần thiết kế chính bằng endpoint, schema, test case và hướng dẫn vận hành.

TC01 - Login thành công

Mục TC01 - Login thành công mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Input: username/password hợp lệ. Expected: nhận access token và refresh token.

TC02 - Login sai mật khẩu

Mục TC02 - Login sai mật khẩu mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Input: password sai. Expected: HTTP 401, không trả token.

TC03 - Thêm sản phẩm vào giỏ hàng

Mục TC03 - Thêm sản phẩm vào giỏ hàng mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

Input: product_id tồn tại, quantity > 0. Expected: cart_item được tạo hoặc cập nhật.

TC04 - Checkout khi hết hàng

Mục TC04 - Checkout khi hết hàng mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

Input: cart có sản phẩm stock = 0. Expected: order không được tạo, trả lỗi 409.

TC05 - Thanh toán thành công

Mục TC05 - Thanh toán thành công mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

Input: order_id hợp lệ, amount đúng. Expected: payment status = success và phát event.

TC06 - Thanh toán thất bại

Mục TC06 - Thanh toán thất bại mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

Input: payment gateway trả failed. Expected: order status = payment_failed.

TC07 - Tạo vận đơn

Mục TC07 - Tạo vận đơn mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

Input: PaymentSucceeded event. Expected: shipment được tạo với status processing.

TC08 - Recommendation

Mục TC08 - Recommendation mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

Input: user có lịch sử view/click. Expected: trả danh sách top-N có score và reason.

TC09 - Chatbot

Mục TC09 - Chatbot mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

Input: câu hỏi tiếng Việt về laptop giá rẻ. Expected: chatbot trả gợi ý có căn cứ sản phẩm.

TC10 - Observability

Mục TC10 - Observability mô tả chi tiết một phần triển khai cần thiết của hệ thống. Nội dung này giúp sinh viên có căn cứ để hiện thực hóa thiết kế trong code, kiểm thử và báo cáo kết quả.

Nội dung

Input: request checkout. Expected: log có trace_id xuyên suốt gateway, order, payment, shipping.

Phụ lục D. Hướng dẫn triển khai mẫu

Phụ lục D. Hướng dẫn triển khai mẫu cung cấp phần minh họa thực hành để tài liệu có thể dùng trực tiếp khi triển khai hoặc thuyết trình. Các nội dung trong phụ lục bổ sung cho phần thiết kế chính bằng endpoint, schema, test case và hướng dẫn vận hành.

Tài liệu tham khảo

1. Eric Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software.
2. Martin Fowler, Patterns of Enterprise Application Architecture.
3. Sam Newman, Building Microservices.
4. Chris Richardson, Microservices Patterns.
5. Django REST framework documentation.
6. FastAPI documentation.
7. Neo4j Cypher Manual.
8. Kubernetes Documentation.
9. Nginx Reverse Proxy Documentation.
10. Tài liệu đề cương môn Kiến trúc và Thiết kế Phần mềm.